AIStudio — API Introduction

Version: 1.0.0

How to drive AIStudio from your own code: the local HTTP API, what it exposes, and the handful of behaviors a client must understand to use it correctly. Written for anyone building an integration on top of AIStudio — including a Model Context Protocol (MCP) server that lets other AI clients use AIStudio as an intelligent searcher over a private document set.

---

1. What the API is

AIStudio runs entirely on your own machine and exposes a local HTTP API. The interface you click is just one client of that API; your code can be another.

- It listens on **localhost only**, with **no authentication** — it is not a remote service, and you should not expose it as one without putting your own gateway in front.
- A **corpus** is the unit of search — a named document set that maps one-to-one to a collection in the vector store.
- The core loop is always the same: *a query is embedded, the most relevant chunks are retrieved (optionally restricted to one firm and re-ranked), and either the chunks are returned as-is or a local model composes an answer over them and cites its sources.*

That gives two natural modes for a client: **retrieve** (get the evidence and reason over it yourself) and **ask** (let AIStudio's own model write the answer).

---

2. The two core calls

Retrieve — get ranked chunks, no model

This is the workhorse for a search/lookup tool: fast, deterministic, and it hands the *evidence* back to your code (or to a calling model) to reason over. It returns ranked chunks without invoking an answer model.

Request fields:

Field	Meaning
`query`	the search text (required)
`corpus`	which corpus to search
`top_k`	how many chunks to return
`hybrid_alpha`	literal ↔ conceptual balance (0 = exact terms, 1 = meaning; omit for the corpus default)
`min_score`	relevance floor; chunks below it are dropped
`entity_filter`	restrict to one or more firms, by display name (e.g. `["BlackRock, Inc."]`)
`allowed_source_paths`	restrict to specific files
`keywords`	extra terms to boost on the literal channel

Each returned chunk carries its score, its source document, and its text. The text begins with a short label — `[Document: <firm> | <ticker> FY<year>]` — that tells you which firm and year the chunk concerns; read the firm from there. (Retrieve does **not** isolate firms on its own — if you want one firm, pass `entity_filter` yourself. See §4.)

Ask — a full answer with citations

Use this when you want AIStudio's local model to compose the answer rather than handing chunks back. It accepts every retrieve field, plus:

Field	Meaning
`model`	which local answer model to use (omit for the active one)
`temperature`	answer-model creativity (low is best for document Q&A)
`entity_filter_mode`	`auto` (default), `yaml`, or `none` — see §4
`query_expansion`	whether to widen the query with the recognized firm's names

It returns a natural-language answer, a list of citations, and the query that was actually run.

---

3. Discovery

Two read-only calls let a client learn what it can search before it searches:

- **List corpora** — every available corpus with its description and settings.
- **Describe a corpus** — one corpus's file list, chunk counts, and routing guidance.

A client (an MCP server, say) would surface these as "list collections" and "describe collection" so the calling model knows the landscape.

---

4. Firm isolation — the one contract to get right

On a multi-firm corpus, the most important behavior is **isolation**: making sure a question about one firm is answered only from that firm's documents. Two things to know:

- **Ask isolates automatically.** With the default mode, *ask* detects the firms named in the question and restricts retrieval to them — no work on your side. Setting the mode to `yaml` uses the firms you pass explicitly; `none` disables isolation. An explicit `entity_filter` always wins, whatever the mode.
- **Retrieve does not.** *Retrieve* honors only the `entity_filter` you send. A firm query with no filter will look "wrong" on retrieve while *ask* resolves it — which is useful for A/B-testing the effect of the filter.

The catch worth internalizing: **a firm the corpus doesn't recognize won't be auto-detected.** If you add documents for a firm that isn't part of the corpus's entity data, its chunks are still tagged and searchable — but automatic detection and isolation will skip it. For such firms, pass `entity_filter` explicitly and isolation works.

Why this matters: without isolation, a firm query falls back to broad similarity, where other firms that discuss the same topic densely crowd out the one you asked about. Isolation is what turns "the right topic, the wrong company" into the right answer.

---

5. Two things every client must handle

- **Citations are advisory.** Different answer models mark references differently, and the count can come back short or doubled. When precision matters, re-derive sources from the *retrieved set* rather than trusting the answer's inline markers.

- **Source references are local file paths.** Citations point at files on the local machine. A client must **not** surface raw paths to a remote caller — map them to filenames, or serve the document through the **source endpoint** (which takes a path and an optional page hint) and expose that instead.

---

6. Supporting endpoints

Beyond retrieve, ask, and discovery, the API covers the full corpus lifecycle:

- **Health and warm-up** — a liveness check, and a *prewarm* call that loads a model into memory. The first call to a cold model is slow (tens of seconds); prewarm before your first ask.
- **Models** — list available models and select the active one.
- **Corpus management** — create, rename, and delete corpora; edit a corpus's description and routing guidance.
- **Ingestion** — upload a file (which does not ingest it), run an ingest pass over a subset or the whole corpus, poll its progress, or cancel it.
- **File maintenance** — verify an indexed file by hash, wipe one file's chunks for re-ingest, or remove a file entirely.
- **Source serving** — fetch a local source document by path and page, for citation resolution.

---

7. A minimal client, sketched

A correct "intelligent corpus searcher" needs only four operations:

1. **list_corpora** → list corpora, so the client knows what's searchable.
2. **describe_corpus(name)** → one corpus's details and routing guidance.
3. **search(query, corpus, firm?, top_k?)** → retrieve. The workhorse. Return chunks as `{firm, text, source_filename, score}` — read `firm` from the chunk's label, and map the source path to a bare filename (never leak the path).
4. **answer(query, corpus, firm?)** → ask. Optional — only when you want AIStudio's model to compose the answer.

Two design notes:

- **Be explicit, not stateful.** The backend tracks an "active model," but a client serving more than one caller should pass the model and parameters on every call rather than rely on shared state.
- **Treat precise table numbers as low-confidence.** Dense numeric tables retrieve poorly; a figure stated in prose is reliable, but a number lifted from a grid should be verified against the chunk it was drawn from before you repeat it.

---

8. The short version

Wrap **retrieve** as your search tool and **ask** as your optional answer tool; use **list/describe** so the caller knows what exists. Pass an explicit firm filter whenever you can't be sure the corpus will recognize the firm. Trust the retrieved set over the model's citation markers. Never leak a local path. Prewarm before the first ask. Get those right and AIStudio behaves as a clean, grounded, fully-local corpus searcher.